You can also give sub and gsub a hash as the replacement parameter, in which case they will look up matched groups and use the corresponding values as replacement text:

```
replacement = { "cat" => "feline", "dog" => "canine" }
replacement.default = "unknown"

"cat and dog".gsub(/\w+/, replacement) # => "feline unknown canine"
```

Backslash Sequences in the Substitution

Earlier we noted that the sequences \1, \2, and so on, are available in the pattern, standing for the _n_th group matched so far. The same sequences can be used in the second argument of sub and gsub.

```
puts "fred:smith".sub(/(\w+):(\w+)/, '\2, \1')
puts "nercpyitno".gsub(/(.)(.)/, '\2\1')
```

produces:

```
smith, fred
encryption
```

You can also reference named groups:

tut_regexp/named_regex_groups_4.rb

```
puts "fred:smith".sub(/(?<first>\w+):(?<last>\w+)/, '\k<last>, \k<first>')
puts "nercpyitno".gsub(/(?<c1>.)(?<c2>.)/, '\k<c2>\k<c1>')
```

produces:

```
smith, fred
encryption
```

Additional backslash sequences work in substitution strings: \& (last match), \+ (last matched group), \` (string prior to match), ' (string after match), and \\ (a literal backslash).

It gets confusing if you want to include a literal backslash in a substitution. Your first attempt might be str.gsub(/\\/, '\\\\').

Clearly, this code is trying to replace each backslash in str with two. The programmer doubled up the backslashes in the replacement text, knowing that they'd be converted to \\ in syntax analysis. However, when the substitution occurs, the regular expression engine performs another pass through the string, converting \\ to \, so the net effect is to replace each single backslash with another single backslash. You need to write gsub(/\\/, '\\\\\\\\')!

```
str = 'a\b\c'               # => "a\b\c"
str.gsub(/\\/, '\\\\\\\\') # => "a\\b\\c"
```

However, using the fact that \& is replaced by the matched string, you could also write this:

```
str = 'a\b\c'          # => "a\b\c"
str.gsub(/\\/, '\&\&') # => "a\\b\\c"
```

If you use the block form of gsub, the string for substitution is analyzed only once (during the syntax pass), and the result is what you intended:

```
str = 'a\b\c'             # => "a\b\c"
str.gsub(/\\/) { '\\\\' } # => "a\\b\\c"
```